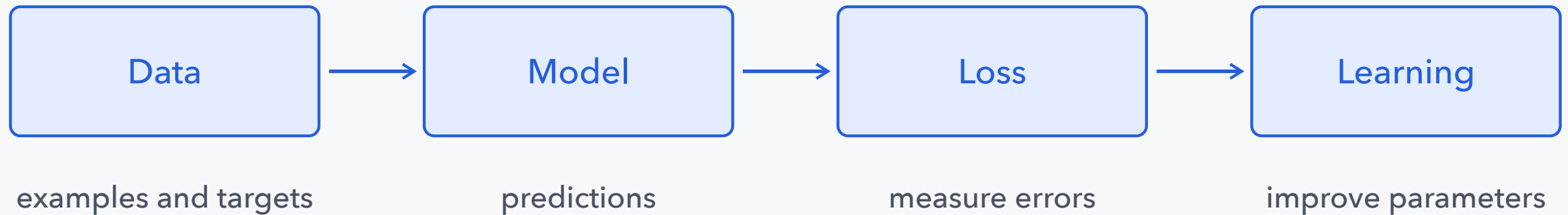


Machine learning: a course recap

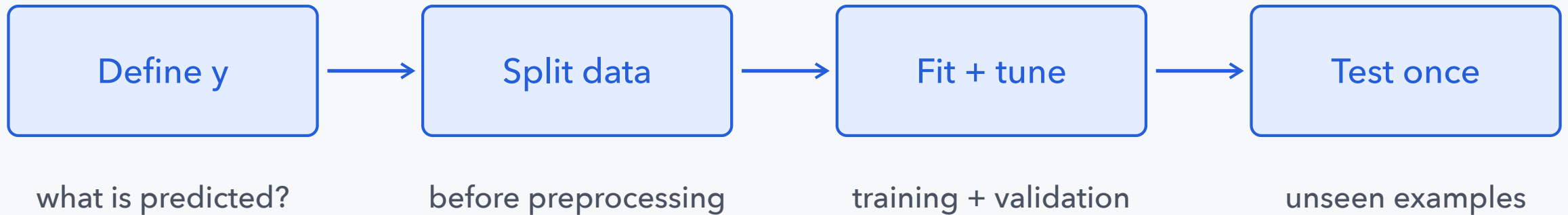


Today's 55-minute route

Minutes	Revision focus
0-7	Data, tasks and evaluation
7-16	Trees, model selection and ensembles
16-28	Regression, optimization and ridge
28-39	Logistic regression and MLPs
39-51	Next-token prediction and autograd
51-55	Synthesis and exit questions

| Worked examples, short predictions and a final diagnosis exercise.

A prediction workflow



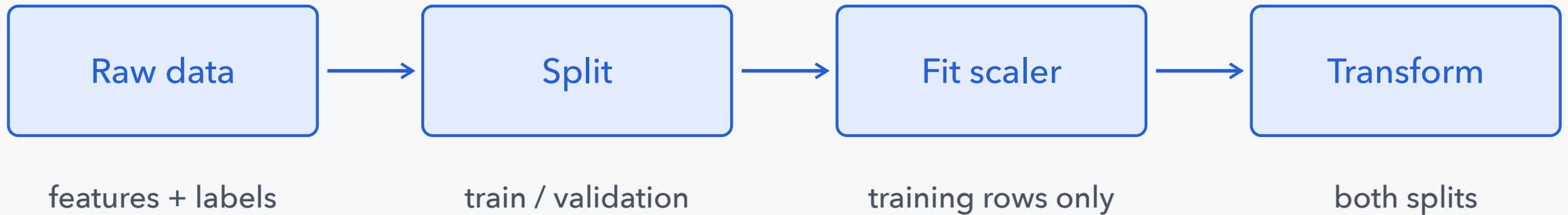
| A good training fit is only one part of a useful predictive model.

Inputs and targets determine the task

Input x	Target y	Output
Tomato measurements	good / bad	class or class probability
House attributes	price	a real number
Five previous characters	next character	probability over characters

The same data can support different tasks if we change the target.

Fit preprocessing inside the training split



Validation information must not influence the fitted preprocessing.

The denominator changes the question

		Predicted label	
		positive	negative
positive	TP = 18	FN = 2	
negative	FP = 12	TN = 68	

$$\text{Precision} = 18 / 30 = 0.60$$

$$\text{Recall} = 18 / 20 = 0.90$$

$$\text{Accuracy} = 86 / 100 = 0.86$$

Actual labels are the rows.

100 illustrative examples

Precision asks about flagged examples; recall asks about actual positives.

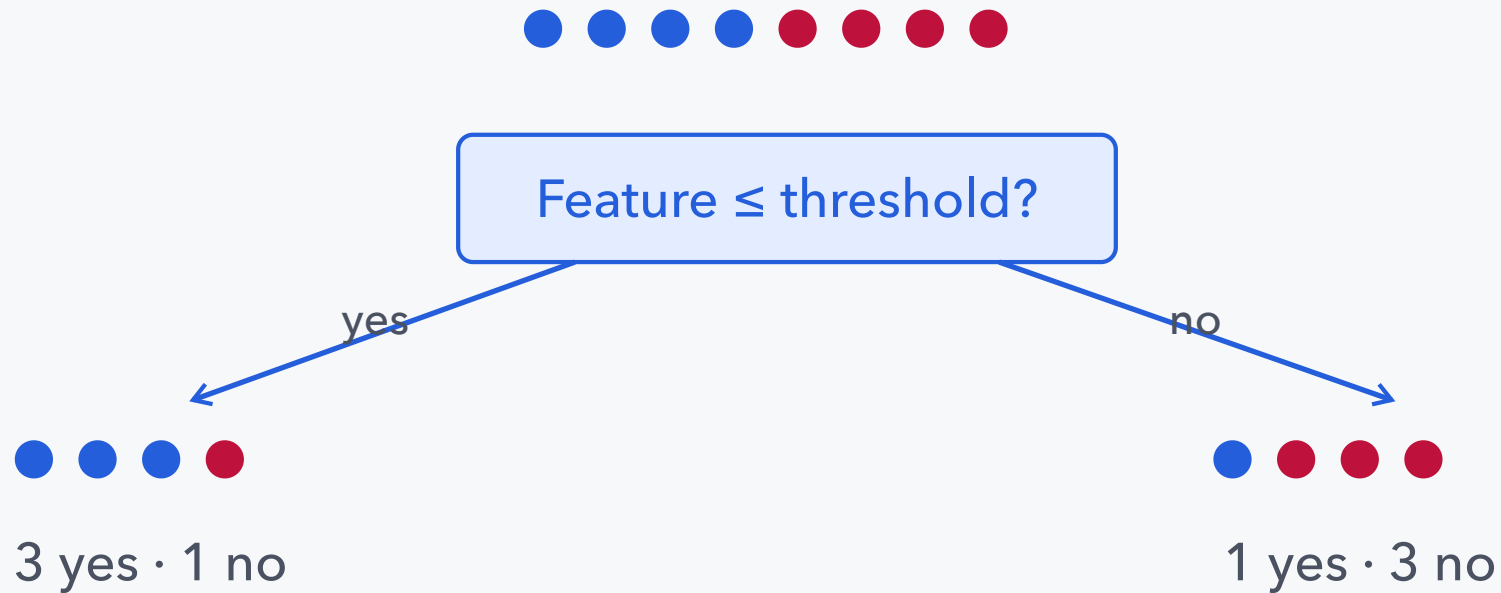
Regression errors should not cancel

Errors $y - \hat{y}$	Mean error	Magnitude-based loss
+5, -5	0	MAE = 5
+5, -5	0	MSE = 25
One error doubles	–	Its squared contribution $\times 4$

The loss determines which mistakes receive the strongest penalty.

A tree partitions the training examples

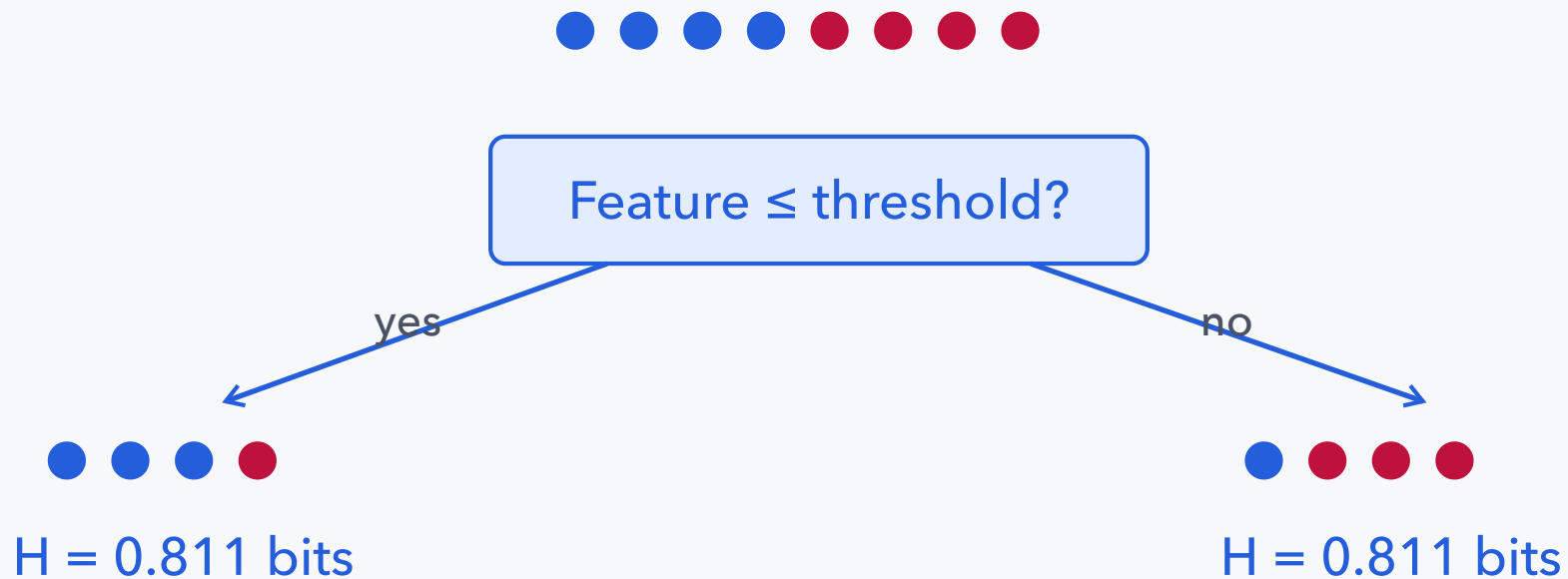
Eight training examples: four yes, four no



Each internal node asks a question; each leaf supplies a prediction.

Information gain weights the children

Eight training examples: four yes, four no



$$\text{Gain} = 1 - (4/8 \times 0.811 + 4/8 \times 0.811) = 0.189 \text{ bits}$$

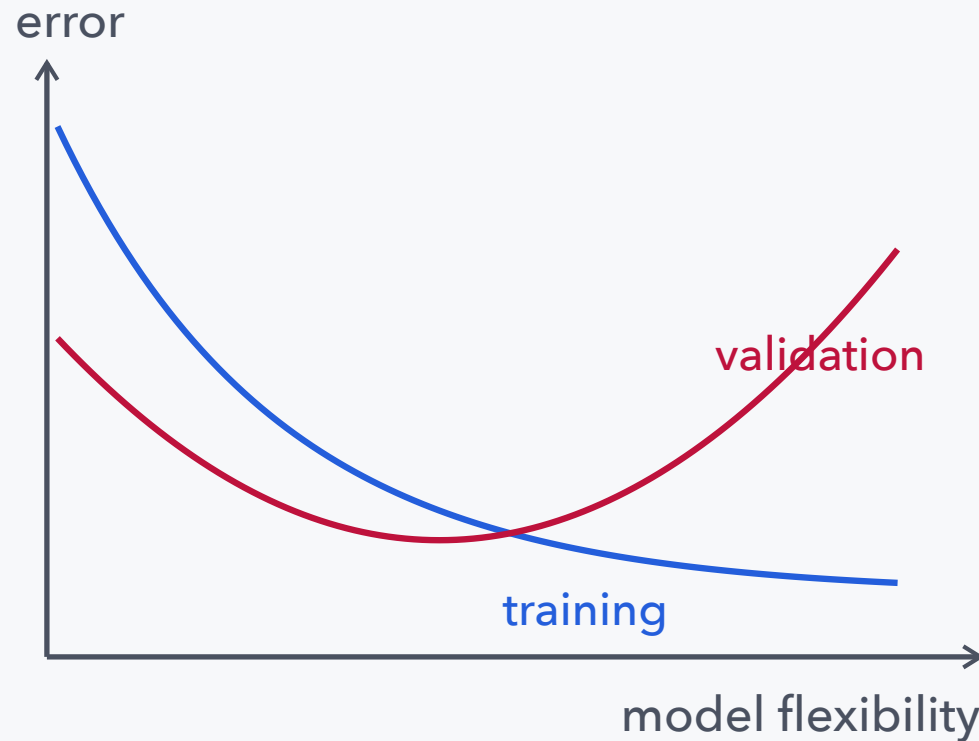
$H(p) = -p \log_2 p - (1-p) \log_2(1-p)$; a pure binary leaf has $H = 0$.

A regression leaf predicts an average

Leaf values (kWh)	Prediction	Squared residuals
30, 32, 40	34 kWh	$16 + 4 + 36 = 56$
Left: 30, 32	31 kWh	$1 + 1 = 2$
Right: 40	40 kWh	0

The mean minimizes squared error within a leaf.

Model flexibility and generalization



Too rigid

Misses the pattern → high bias

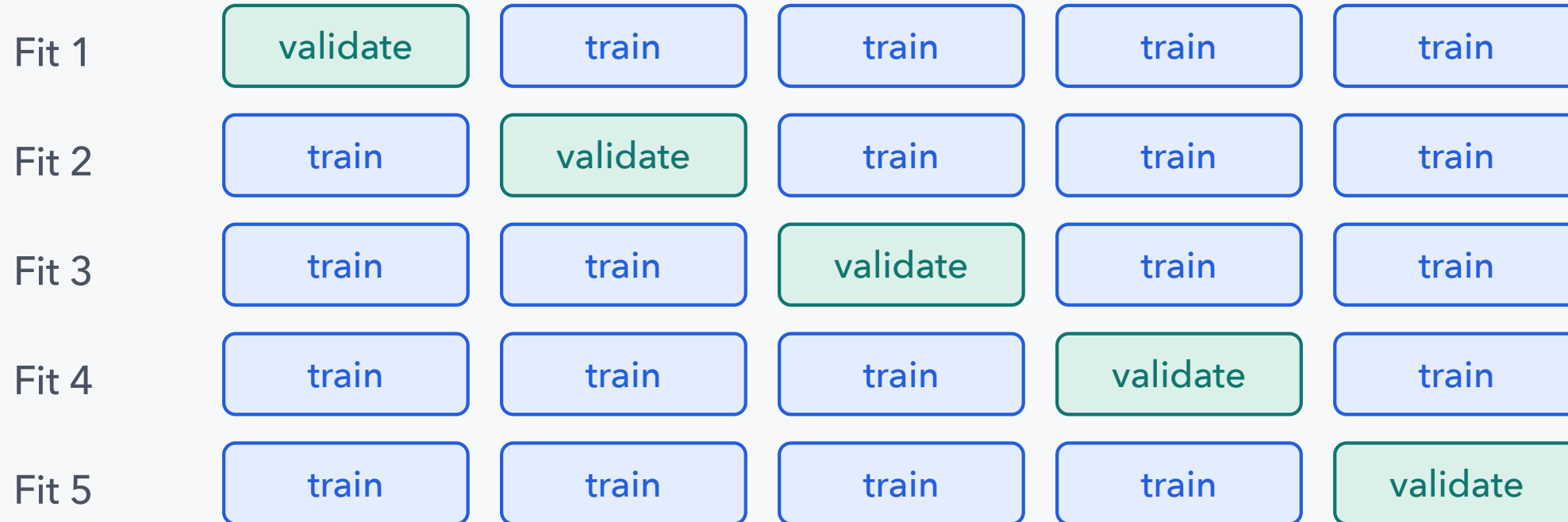
Too flexible

Changes with the sample → high variance

Choose using held-out performance.

Schematic curves: choose flexibility using validation performance.

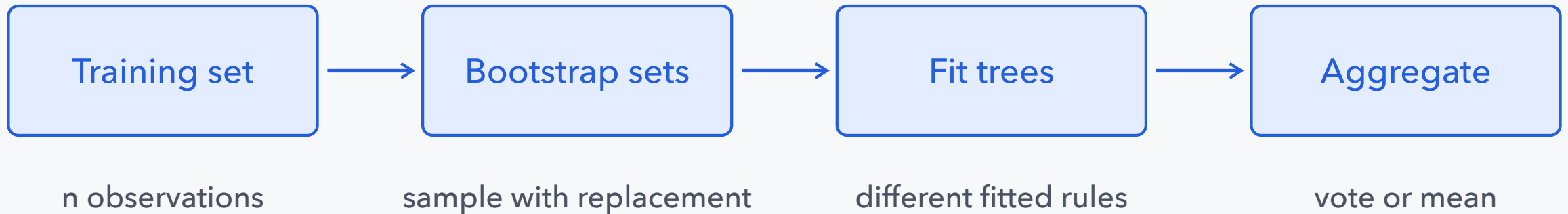
Cross-validation reuses data carefully



Select settings from mean validation score; test once at the end.

For each candidate setting, repeat the whole fitting pipeline in every fold.

Bagging averages models fitted to resamples



Averaging helps most when individual errors are not perfectly correlated.

Random forests and boosting

Method	How models differ	How predictions combine
Bagging	Bootstrap training sets	Vote / average
Random forest	Bootstrap + feature subsets	Vote / average
Boosting	Fit sequentially to current errors	Weighted / additive model

Random forests reduce similarity between trees; boosting builds sequentially.

Choose the model without using the test set

Candidate depth	Training accuracy	Validation accuracy
2	0.81	0.80
6	0.95	0.88
Unlimited	1.00	0.82

Illustrative validation results; the test set has not been opened.

Fit a line to three observations



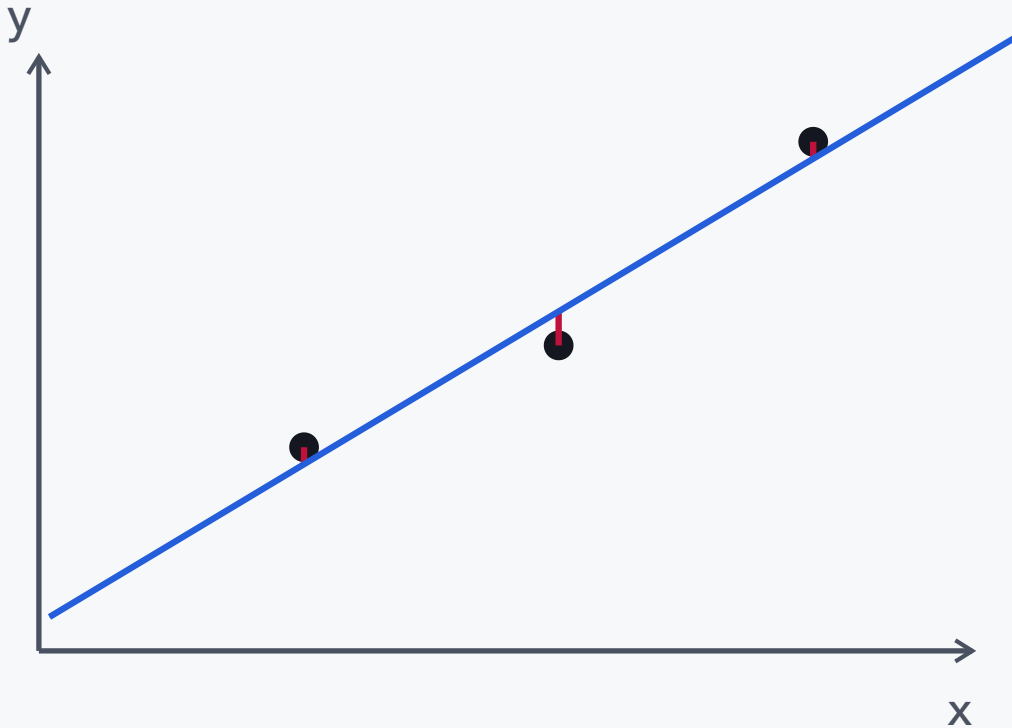
$$\hat{y} = w x + b$$

Data: (1, 2), (2, 3), (3, 5)

Find one slope and one intercept.

Two parameters describe the rule; the data determine their fitted values.

Least squares balances all residuals



$$\hat{y} = w x + b$$

Data: (1, 2), (2, 3), (3, 5)

$$w = 1.5 \quad b = 1/3$$

Residuals: $1/6$, $-1/3$, $1/6$

$$SSE = 1/6 \quad MSE = 1/18$$

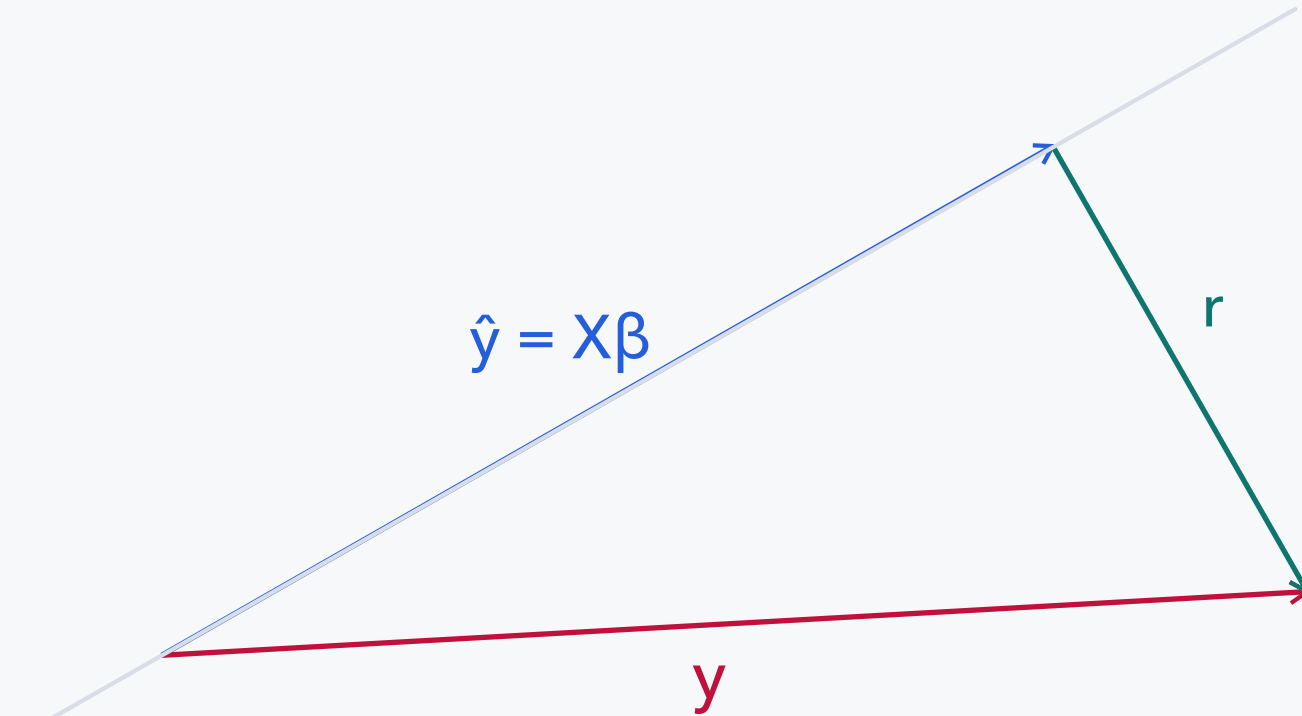
The least-squares line need not pass through any training point.

The normal equation

Object	Shape	Meaning
X (includes a ones column)	$n \times (d+1)$	Inputs + intercept feature
β	$(d+1) \times 1$	Coefficients
$\hat{y} = X\beta$	$n \times 1$	Fitted targets
$X^T X \beta = X^T y$	$(d+1) \times 1$	Zero-gradient condition

If X has full column rank: $\beta = (X^T X)^{-1} X^T y$.

Least squares as a projection



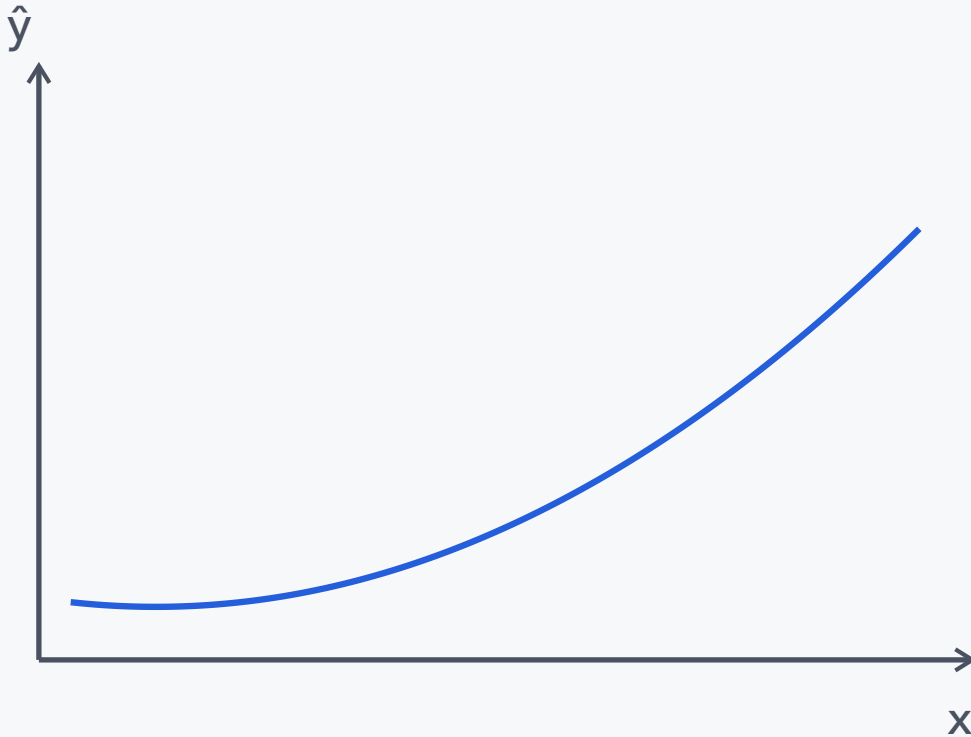
Schematic: fitted vector lies in the column space of X .

$$X^T r = 0$$

Residual is orthogonal
to every column of X .

The fitted vector is the closest point to y within the column space of X .

Linear in the parameters can still make a curve



$$x \rightarrow [1, x, x^2]$$

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$$

Curved in x

Linear in $\beta_0, \beta_1, \beta_2$

Choose the basis using validation.

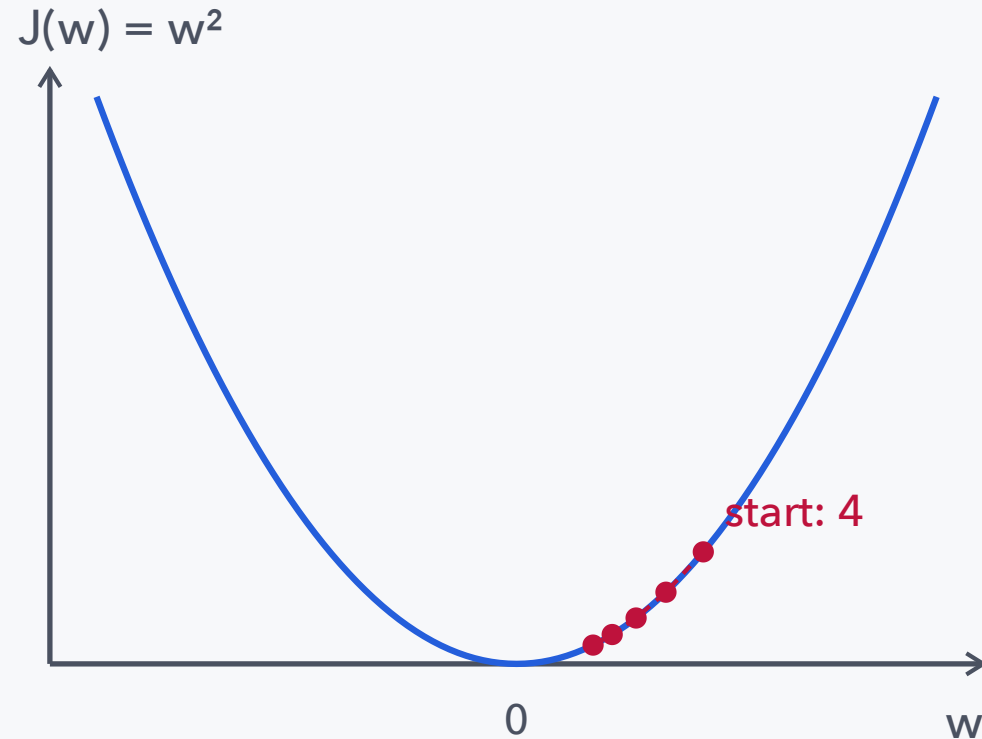
Basis functions change the representation; least squares can remain the fitter.

Different coefficients can make the same prediction

Features	Coefficients	Prediction
$x_1 = x_2 = x$	$w_1 = 2, w_2 = 0$	$2x$
$x_1 = x_2 = x$	$w_1 = 1, w_2 = 1$	$2x$
$x_1 = x_2 = x$	$w_1 = 4, w_2 = -2$	$2x$

When features repeat information, individual coefficients can be unstable.

Gradient descent takes a local step



$$w \leftarrow w - \eta \cdot 2w$$

$$\eta = 0.1$$

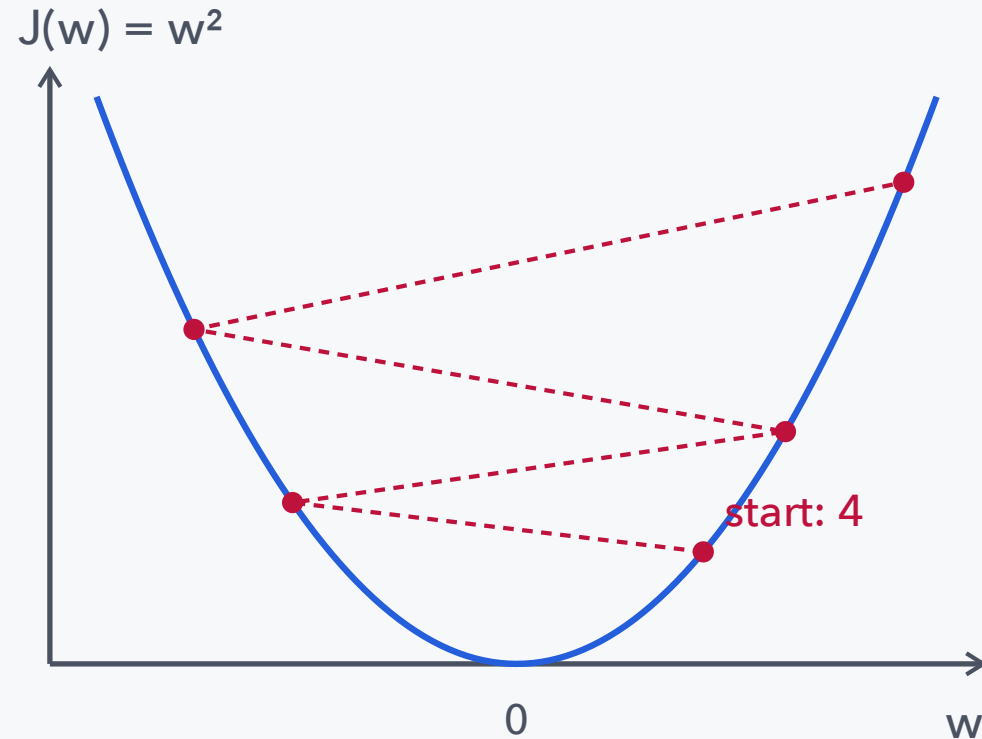
$$4.00 \rightarrow 3.20 \rightarrow 2.56$$

$$2.05 \rightarrow 1.64$$

Red dots: successive parameter values.

For a differentiable objective: $\theta \leftarrow \theta - \eta \nabla J(\theta)$.

The learning rate can change the outcome



$$w \leftarrow w - \eta \cdot 2w$$

$$\eta = 1.1$$

$$4.00 \rightarrow -4.80 \rightarrow 5.76$$

$$-6.91 \rightarrow 8.29$$

Red dots: successive parameter values.

For this quadratic, $w_{t+1} = (1 - 2\eta)w_t$; convergence requires $0 < \eta < 1$.

How much data supplies one gradient?

Method	Examples per update	Trade-off
Full batch	All training examples	Accurate full-data gradient
Stochastic	One sampled example	Noisy, inexpensive updates
Mini-batch	A subset of examples	Efficient vectorized updates

| A stochastic update need not reduce the loss over the entire dataset.

Ridge trades training fit for smaller coefficients

Term	Role
$(1/n) \sum (y_i - \hat{y}_i)^2$	Fit the training targets
$\lambda \sum_j w_j^2$	Penalize large feature coefficients
Choose λ using validation	Control the strength of shrinkage
Scale using training data	Make feature units comparable

This convention uses mean squared error; the intercept is not penalized.

Ridge prefers sharing the weight

Constraint: $w_1 + w_2 = 2$	Same predictions	Penalty $w_1^2 + w_2^2$
(2, 0)	2x	4
(1, 1)	2x	2
(4, -2)	2x	20

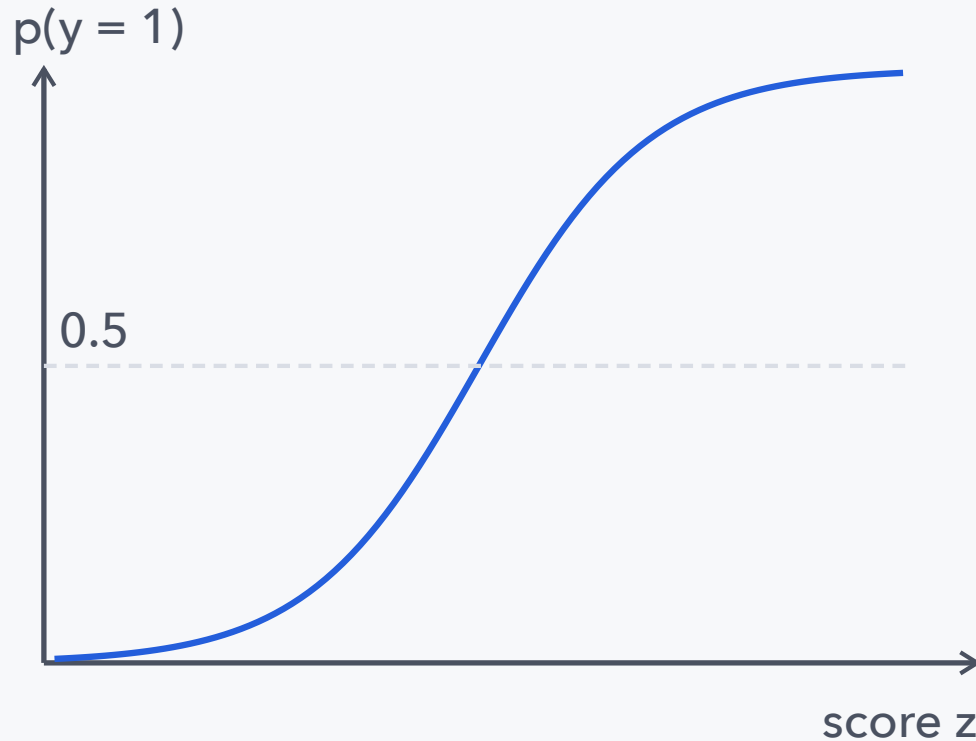
Among these equally fitting choices, ridge prefers (1, 1).

Three ways to control a regression model

Model	Representation	A setting to validate
Regression tree	Regions and leaf means	Depth / minimum leaf size
Basis regression	Chosen feature functions	Basis complexity
Ridge regression	Weighted feature sum	Penalty λ

| The model, representation and fitting objective are separate choices.

A linear score becomes a class probability



$$z = \mathbf{w}^T \mathbf{x} + b$$

$$p = 1 / (1 + \exp(-z))$$

$$z = 0 \rightarrow p = 0.5$$

$$z \approx 2.20 \rightarrow p \approx 0.90$$

Logistic regression uses a sigmoid output for a binary target.

Cross-entropy penalizes confident mistakes

True label $y = 1$	Probability p	Loss $-\ln p$
Confident and correct	0.9	0.105
Uncertain	0.5	0.693
Confident and wrong	0.1	2.303

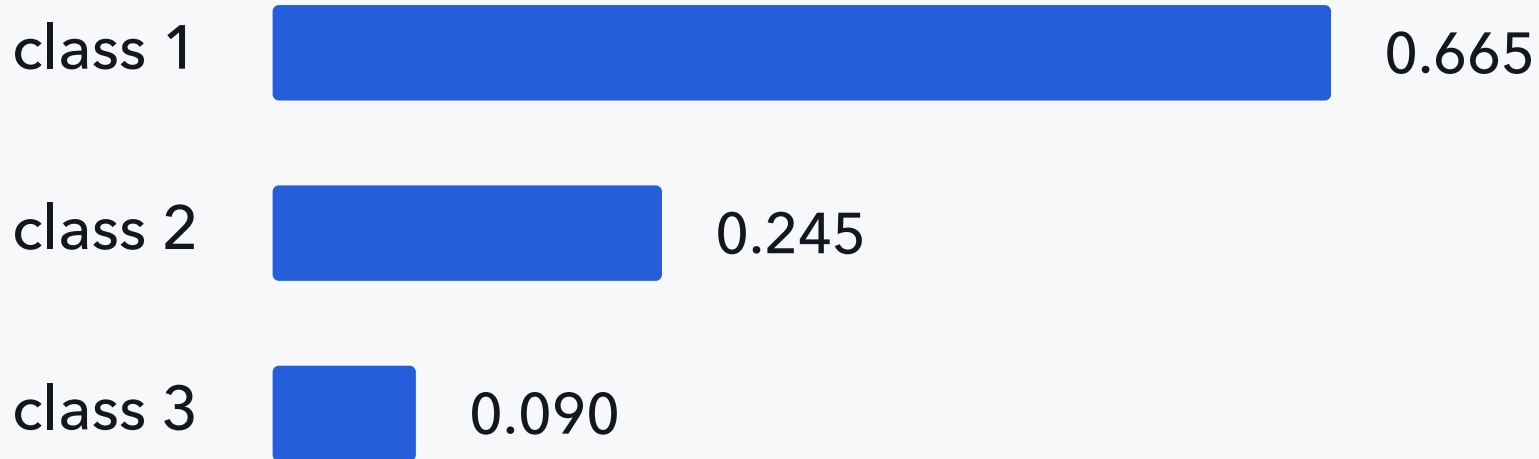
Binary loss: $-[y \ln p + (1-y) \ln(1-p)]$.

A threshold turns probabilities into decisions

Rule	Boundary in score z	Effect of increasing threshold
Predict 1 when $p \geq 0.5$	$z \geq 0$	Default example
Predict 1 when $p \geq 0.8$	$z \geq \ln(4) \approx 1.386$	Fewer positive predictions
On the same scored examples	Move the cutoff	Recall cannot increase

Threshold choice reflects the costs of false positives and false negatives.

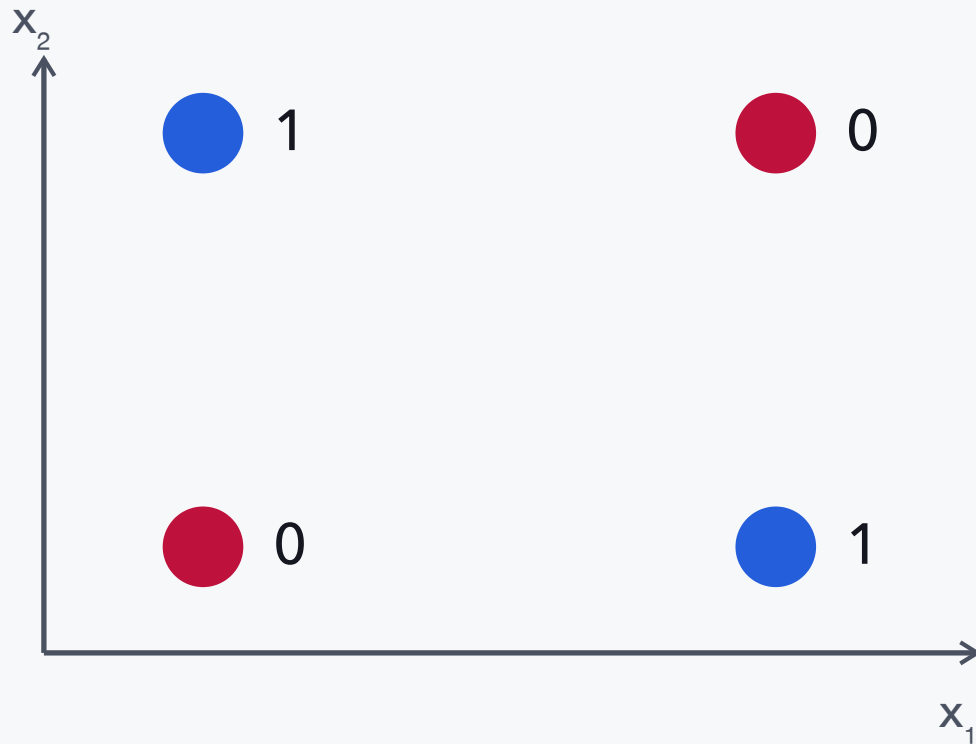
One score per class



Illustrative probabilities; not predictions from a trained model.

For scores $[2, 1, 0]$, softmax gives $\exp(z_k) / \sum_j \exp(z_j)$.

One linear boundary cannot separate XOR

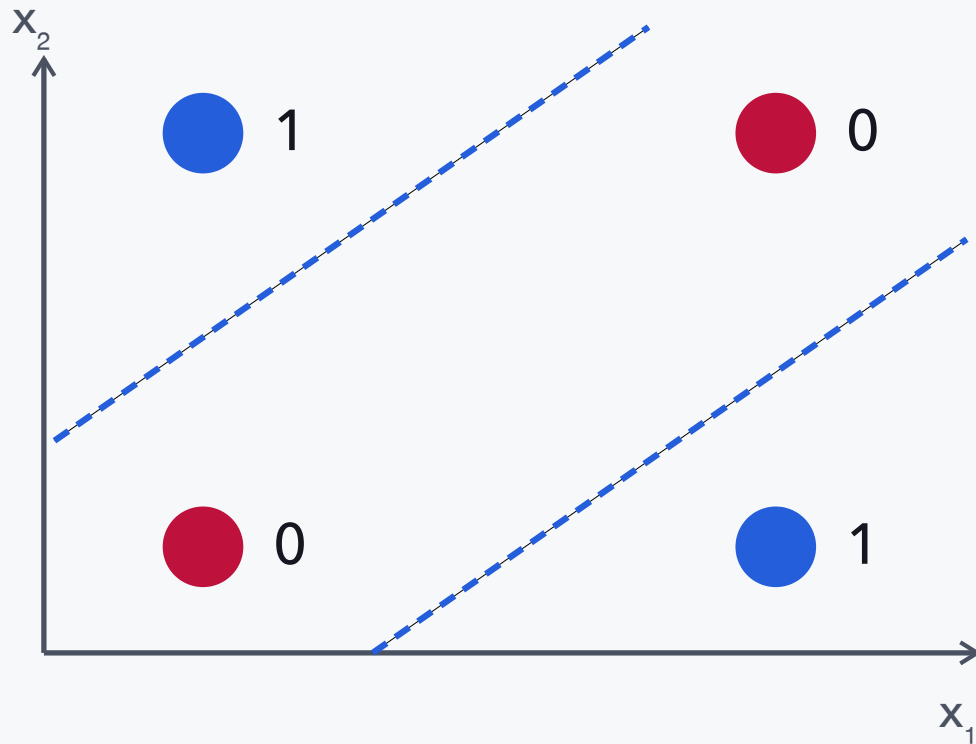


Opposite corners share a class.

One straight boundary fails.

The obstacle is the representation, not just how long we optimize.

Two ReLU features solve the four XOR corners



$$h_1 = \text{ReLU}(x_1 + x_2)$$

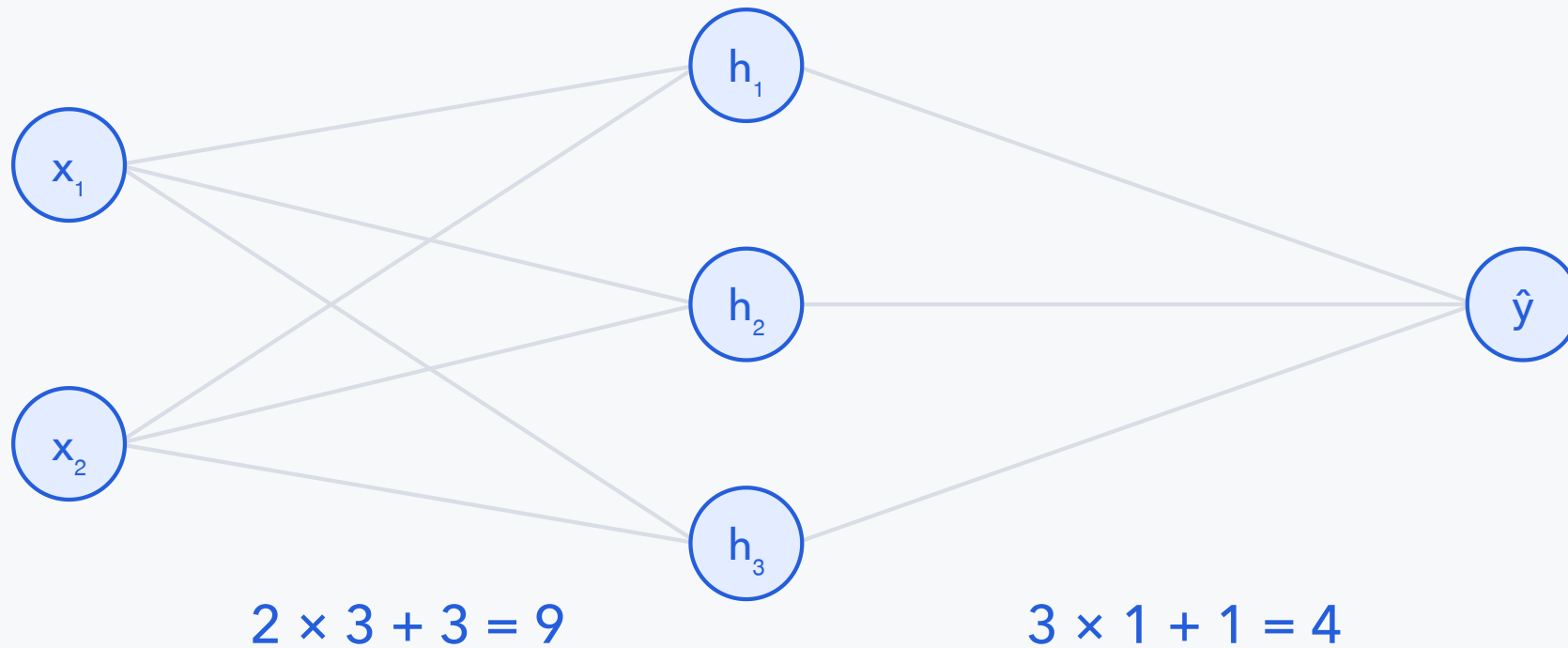
$$h_2 = \text{ReLU}(x_1 + x_2 - 1)$$

$$\hat{y} = h_1 - 2h_2$$

Outputs at the four corners: 0, 1, 1, 0

For binary inputs, $\hat{y} = \text{ReLU}(x_1 + x_2) - 2 \text{ReLU}(x_1 + x_2 - 1)$.

Count weights and biases by layer



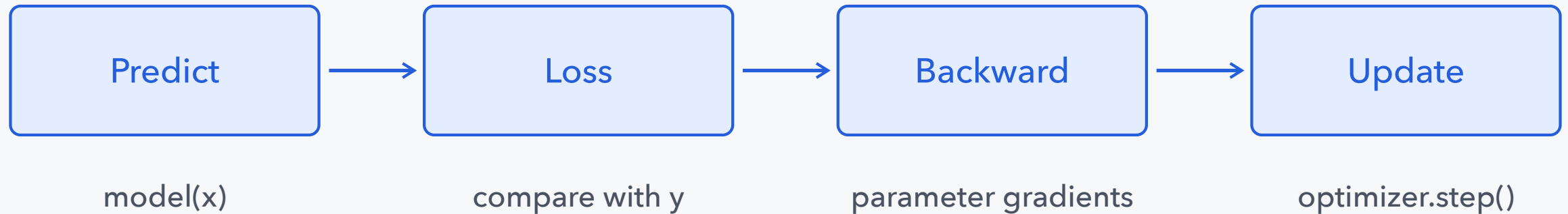
A dense $a \rightarrow b$ layer has $a \times b$ weights and b biases.

Nonlinearities prevent affine layers collapsing

Construction	Equivalent form	Consequence
$W_2(W_1x+b_1)+b_2$	$Ax+c$	Still affine
$W_2\varphi(W_1x+b_1)+b_2$	Learned nonlinear features	Can bend the decision boundary
$\varphi = \text{ReLU, tanh, or another choice}$	Activation affects behavior	Choose for the model and task

The output layer and loss must still match the target.

The same training loop fits a line or an MLP



Clear accumulated gradients before the backward pass for a fresh batch.

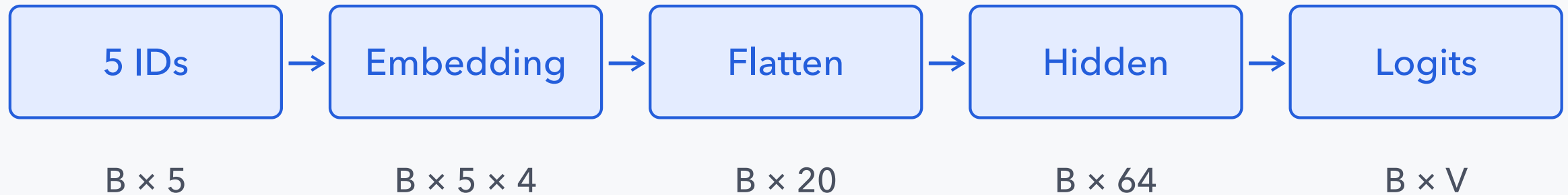
Turn one name into supervised examples

Context (5 characters)	Target
.....	n
....n	i
...ni	p
..nip	u
.nipu	n
nipun	.(end)

Five-character context, as in the names notebook; "." marks padding and end.

Embeddings feed an ordinary MLP

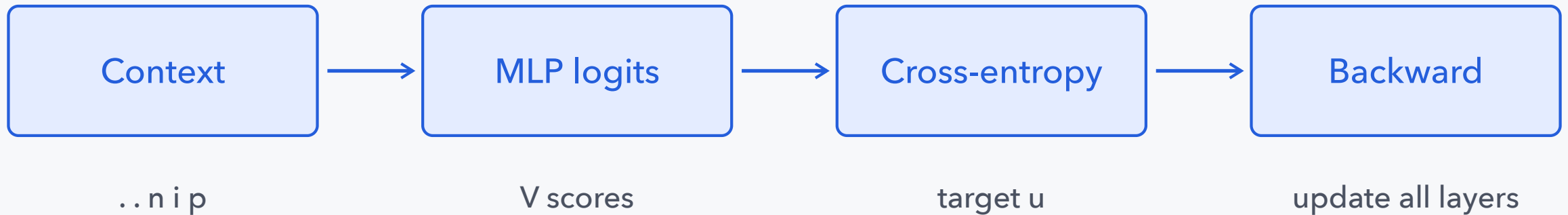
A next-character model is a classifier at every step



V is the vocabulary size; the embedding table is learned.

| Context length 5 · embedding width 4 · hidden width 64 · V output scores.

Training knows the next character



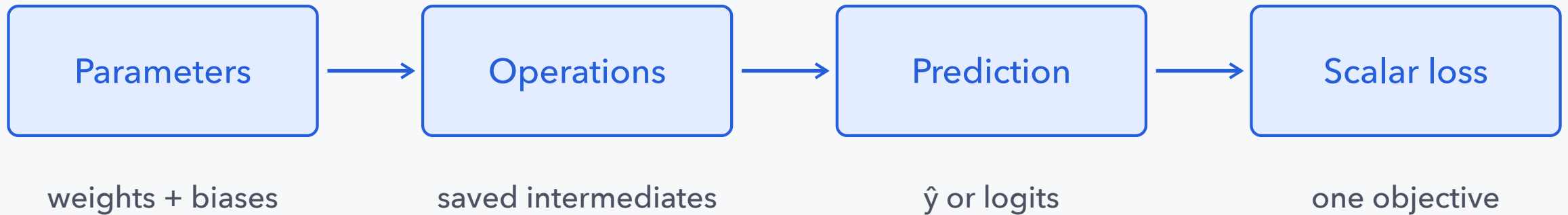
– $\ln p(\text{true next character})$ trains embeddings and classifier weights together.

Generation feeds the chosen character back

Current context	Choose next character	New context
.....	n	n
.....n	i	...ni
...ni	p	..nip
...	...	...
nipun	.	Stop

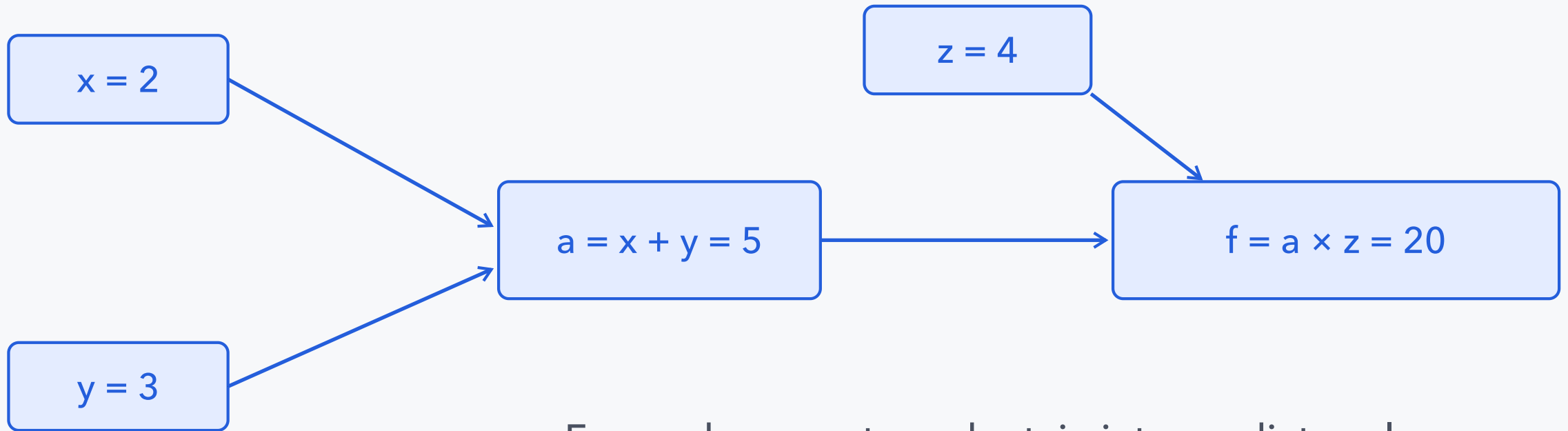
Illustrative trace: predict a distribution → select → append → repeat.

The loss depends on parameters through a graph



Autograd follows the computation that was executed.

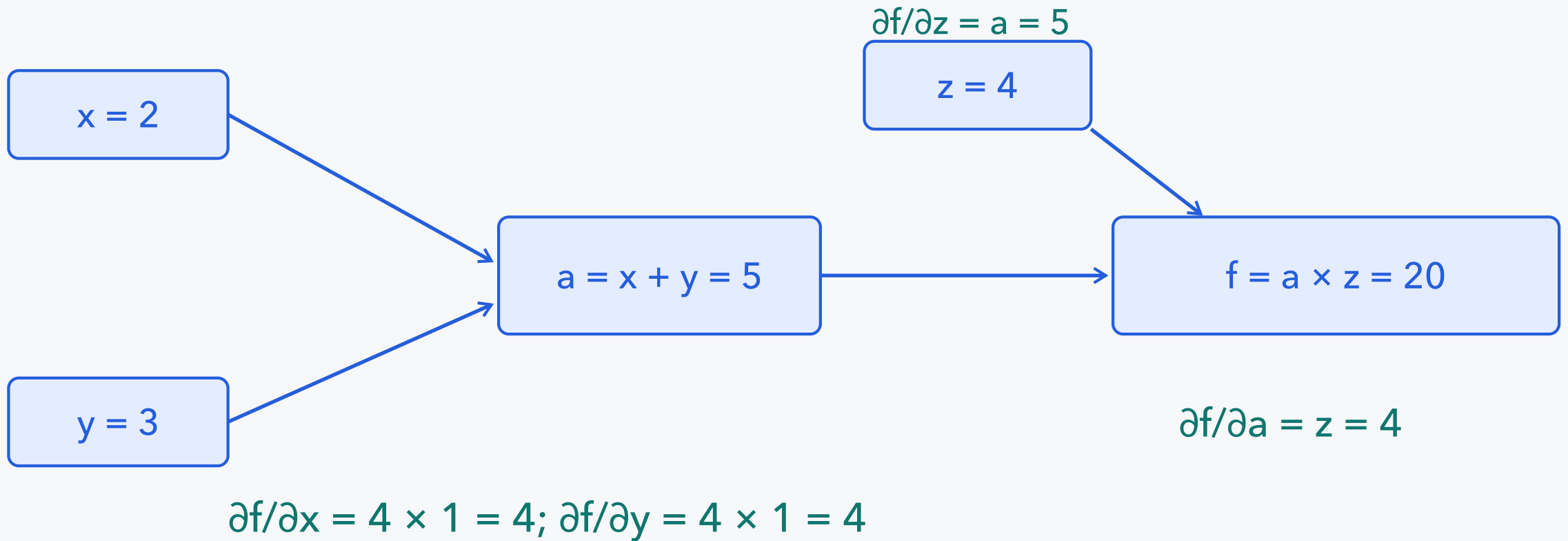
A tiny graph: $f = (x + y)z$



Forward: compute and retain intermediate values.

At $x=2, y=3, z=4$: $a=5$ and $f=20$.

Multiply by a local derivative, then pass it back



Incoming gradient $\times$ local derivative; add contributions at a shared input.

A reused value receives both contributions

Each use contributes a gradient; add both.



Two paths to x

$$\partial f / \partial x = x + x = 6$$

For $f=x \times x$, storing only one incoming contribution gives the wrong derivative.

Automatic differentiation is not finite differencing

Method	How the derivative is obtained	Main limitation
Finite differences	Perturb inputs and re-evaluate	Step size, cancellation, cost
Symbolic differentiation	Manipulate expressions	Expression growth / representation
Reverse-mode autodiff	Local rules in reverse graph order	Must retain or recover intermediates

For a scalar loss, one reverse traversal yields gradients for all its parameters.

Three operations with different jobs

PyTorch call	Job	Common mistake
<code>optimizer.zero_grad()</code>	Clear stored parameter gradients	Accidental batch accumulation
<code>loss.backward()</code>	Compute / accumulate gradients	Expecting weights to change
<code>optimizer.step()</code>	Update model parameters	Updating before backward
<code>torch.no_grad()</code>	Disable graph recording in its scope	Confusing it with <code>model.eval()</code>

| `model.eval()` changes training-dependent modules; it does not disable gradients.

The models in this course

Model	Output / loss	How fitting works
Tree / forest	Class or real value / impurity	Greedy splits + aggregation
Linear / ridge	Real value / squared error (+ penalty)	Solver or gradient updates
Logistic / MLP classifier	Probabilities / cross-entropy	Gradient updates
Name model	Next-character distribution / CE	MLP + learned embeddings

The target, representation, objective and evaluation protocol remain explicit.

Diagnose the mistake before choosing a bigger model

Observation

Training 100%, validation poor

Huge opposite regression weights

MLP behaves like a straight line

Loss explodes after an update

Names repeat training examples

First question

Too much flexibility, or leakage in the protocol?

Correlated or duplicate feature columns?

Were the nonlinear activations omitted?

Learning rate, scaling or gradient problem?

Was evaluation split by name before windows?

Use the smallest check that distinguishes the competing explanations.

Five questions for any predictive model

Question

Recall the connection

What exactly is the target?

Regression, classification, next character

Where does flexibility come from?

Depth, basis functions, hidden features

What is being minimized?

Squared error, impurity, cross-entropy

How do parameters change?

Splits, a solver, gradient + optimizer

What evidence shows it generalizes?

A sound split and held-out evaluation

Define the target, fit the model, and test it on data it has not seen.